

Algorytm Little'a dla problemu komiwojażera

Grupa: Tymoteusz Prostek, Maksymilian Zych, Michał Wójcik

Zadanie 1 — implementacja algorytmu Little'a

Zaimplementowano algorytm Little'a dla problemu komiwojażera.

W implementacji zapisano najważniejsze informacje o podproblemach: numer PP, dolne ograniczenie **LB**, listę aktualnie wybranych odcinków oraz kryterium zamknięcia **KZ**.

Zastosowano następujące oznaczenia:

- **KZ0** — podproblem nie został zamknięty i wykonano jego podział,
- **KZ1** — podproblem jest sprzeczny albo nie można go dalej rozwijać,
- **KZ2** — podproblem zamknięto, ponieważ $LB \geq v^*$,
- **KZ3** — znaleziono pełny cykl Hamiltona.

```
import heapq
import numpy as np

def matrix_reduction(matrix):
    """
    Dokonuje redukcji macierzy kosztów dla Algorytmu Little'a.
    Odejmuje minima z wierszy oraz kolumn i sumuje je, wyznaczając
    dolne ograniczenie.
    """
    inf = float("inf")
    m = matrix.copy()
    reduction_cost = 0.0

    # Redukcja wierszy.
    for i in range(m.shape[0]):
        row_min = np.min(m[i])

        # Jeżeli cały wiersz jest zablokowany, nie wykonuje się
        # redukcji.
        if row_min == inf:
            continue

        # Jeżeli minimum jest większe od zera, odejmuje się je od
        # całego wiersza.
        if row_min > 0:
            reduction_cost += row_min
            m[i] -= row_min

    # Redukcja kolumn.
    for j in range(m.shape[1]):
```

```

        col_min = np.min(m[:, j])

        # Jeżeli cała kolumna jest zablokowana, nie wykonuje się
        redukcji.
        if col_min == inf:
            continue

        # Jeżeli minimum jest większe od zera, odejmuje się je od
        całej kolumny.
        if col_min > 0:
            reduction_cost += col_min
            m[:, j] -= col_min

    return m, reduction_cost

def min_without(values, forbidden_index):
    """
    Wyznacza minimum z tablicy z pominięciem jednego indeksu.
    Funkcja jest potrzebna do liczenia kary dla zera w macierzy.
    """
    inf = float("inf")
    best = inf

    for index, value in enumerate(values):
        if index == forbidden_index:
            continue

        if value < best:
            best = value

    # Jeżeli nie znaleziono żadnej skończonej wartości, przyjmuje się
    0.
    if best == inf:
        return 0.0

    return best

def section_ij(matrix):
    """
    Wyznacza odcinek <i*, j*> o największym optymistycznym koszcie
    wyłączenia.
    Sprawdzane są wszystkie zera w macierzy zredukowanej.
    """
    best_i = None
    best_j = None
    best_penalty = -1.0

    n = matrix.shape[0]

```

```

for i in range(n):
    for j in range(n):
        if np.isclose(matrix[i, j], 0.0):
            row_min = min_without(matrix[i, :], j)
            col_min = min_without(matrix[:, j], i)
            penalty = row_min + col_min

            if penalty > best_penalty:
                best_i = i
                best_j = j
                best_penalty = penalty

    return best_i, best_j, best_penalty

def path_to_one_based(path):
    """
    Zamienia numerację Pythonową 0..n-1 na normalną numerację miast
    1..n.
    """
    return [(i + 1, j + 1) for i, j in path]

def is_full_tour(path, n):
    """
    Sprawdza, czy wybrane odcinki tworzą jeden pełny cykl Hamiltona.
    """
    if len(path) != n:
        return False

    next_city = dict(path)

    # Każde miasto musi mieć dokładnie jedno wyjście.
    if len(next_city) != n:
        return False

    # Każde miasto musi mieć dokładnie jedno wejście.
    if len(set(next_city.values())) != n:
        return False

    start = path[0][0]
    current = start

    for _ in range(n):
        if current not in next_city:
            return False

        current = next_city[current]

    return current == start

```

```

def creates_forbidden_subtour(path, n):
    """
    Sprawdza, czy wybrane odcinki tworzą za wcześnie mały podcykl.
    Mały podcykl jest zabroniony, jeżeli nie obejmuje jeszcze
    wszystkich miast.
    """
    next_city = dict(path)

    for start in next_city:
        current = start
        visited = set()

        while current in next_city:
            if current in visited:
                return not is_full_tour(path, n)

            visited.add(current)
            current = next_city[current]

    return False


def path_cost(original_matrix, path):
    """
    Oblicza rzeczywisty koszt trasy na podstawie oryginalnej macierzy
    kosztów.
    """
    cost = 0.0

    for i, j in path:
        cost += original_matrix[i, j]

    return cost


def order_path_edges(path, n, start=0):
    """
    Porządkuje odcinki tak, aby tworzyły czytelną trasę od wybranego
    miasta startowego.
    """
    next_city = dict(path)

    if start not in next_city:
        start = path[0][0]

    ordered_path = []
    current = start

    for _ in range(n):

```

```

        next_node = next_city[current]
        ordered_path.append((current, next_node))
        current = next_node

    return ordered_path

def split_problem(
    matrix,
    lower_bound,
    path,
    i,
    j,
):
    """
    Dzieli problem na dwa podproblemy:
    1. odcinek i -> j zostaje włączony,
    2. odcinek i -> j zostaje zabroniony.
    """
    inf = float("inf")
    n = matrix.shape[0]
    children = []

    # PP 1: wybrano odcinek i -> j.
    included_path = path + [(i, j)]

    if not creates_forbidden_subtour(included_path, n):
        included_matrix = matrix.copy()

        # Jeżeli wybrano i -> j, to z miasta i nie wolno już wyjechać
        inaczej.
        included_matrix[i, :] = inf

        # Jeżeli wybrano i -> j, to do miasta j nie wolno już wjechać
        inaczej.
        included_matrix[:, j] = inf

        # Blokowany jest natychmiastowy powrót j -> i.
        included_matrix[j, i] = inf

        included_matrix, reduction_cost =
matrix_reduction(included_matrix)
        included_bound = lower_bound + reduction_cost

        children.append((
            included_bound,
            included_matrix,
            included_path,
            f"wybrano odcinek {i + 1}->{j + 1}",

```

```

    ))

    # PP 2: zabroniono odcinka i -> j.

    excluded_matrix = matrix.copy()

    # Zabronienie odcinka realizowane jest przez wpisanie
    nieskończoności.
    excluded_matrix[i, j] = inf

    excluded_matrix, reduction_cost =
matrix_reduction(excluded_matrix)
    excluded_bound = lower_bound + reduction_cost

    children.append((
        excluded_bound,
        excluded_matrix,
        path.copy(),
        f"zabroniono odcinka {i + 1}->{j + 1}",
    ))

    return children

def little_algorithm(cost_matrix):
    """
    Realizuje algorytm Little'a dla problemu TSP.
    Zwracany jest najlepszy koszt, najlepsza trasa oraz lista
    przeanalizowanych PP.
    """
    original_matrix = np.asarray(cost_matrix, dtype=float)
    n = original_matrix.shape[0]

    reduced_matrix, start_reduction =
matrix_reduction(original_matrix)

    v_star = float("inf")
    best_path = []

    queue = []
    counter = 0
    problem_id = 1
    trace = []

    # Na kolejce przechowywane są niezamknięte podproblemy.
    heapq.heappush(queue, (
        start_reduction,
        counter,
        problem_id,
        reduced_matrix,

```

```

        [],
        "problem początkowy",
    ))

    while queue:
        lower_bound, _, current_id, matrix, path, description =
            heapq.heappop(queue)

        chosen_i = None
        chosen_j = None
        chosen_penalty = None

        # KZ2: gałąź nie może poprawić aktualnej wartości odcinającej.
        if lower_bound >= v_star:
            kz = "KZ2"
            decision = "zamknięto, bo LB >= v*"

        # KZ3: znaleziono pełny cykl Hamiltona.
        elif len(path) == n and is_full_tour(path, n):
            current_cost = path_cost(original_matrix, path)

            if current_cost < v_star:
                v_star = current_cost
                best_path = path.copy()

            kz = "KZ3"
            decision = "zamknięto, bo otrzymano pełny cykl"

        else:
            chosen_i, chosen_j, chosen_penalty = section_ij(matrix)

            # KZ1: brak możliwego dalszego wyboru.
            if chosen_i is None or chosen_j is None:
                kz = "KZ1"
                decision = "zamknięto, bo brak dalszych odcinków"
            else:
                kz = "KZ0"
                decision = f"podział względem odcinka {chosen_i + 1}-
>{chosen_j + 1}"

            children = split_problem(matrix, lower_bound, path,
                                    chosen_i, chosen_j)

            for child_lb, child_matrix, child_path,
                child_description in children:
                if child_lb < v_star:
                    counter += 1
                    problem_id += 1

                    heapq.heappush(queue, (

```

```

        child_lb,
        counter,
        problem_id,
        child_matrix,
        child_path,
        child_description,
    ))

    trace.append({
        "PP": current_id,
        "opis": description,
        "LB": lower_bound,
        "KZ": kz,
        "wybrane odcinki": path_to_one_based(path),
        "decyzja": decision,
        "odcinek podziału": None if chosen_i is None else
f"{chosen_i + 1}->{chosen_j + 1}",
        "kara": chosen_penalty,
        "v* po analizie": v_star,
    })

    ordered_best_path = order_path_edges(best_path, n, start=0)
    return v_star, path_to_one_based(ordered_best_path), trace

```

Zadanie 2 — uruchomienie algorytmu dla danych 6x6

Wykonano obliczenia dla przykładowej macierzy kosztów.

Wartości `inf` na przekątnej oznaczają, że przejście z miasta do samego siebie jest zabronione.

```

def format_edges(edges):
    """
    Zamienia listę odcinków na prosty tekst do tabeli.
    """
    if not edges:
        return "-"

    return ", ".join([f"{i}->{j}" for i, j in edges])

def format_value(value):
    """
    Ładnie zapisuje liczby, INF oraz wartości puste.
    """
    if value is None:
        return "-"

    if value == float("inf"):
        return "INF"

```

```

        return f"{float(value):.1f}"

def print_trace_table(trace):
    """
    Wypisuje kolejne przeanalizowane podproblemy PP.
    """
    header = f"{'PP':>3} | {'LB':>6} | {'KZ':>4} | {'odcinki':<35} |  
{'decyzja'}"
    print(header)
    print("-" * len(header))

    for row in trace:
        pp = row["PP"]
        lb = format_value(row["LB"])
        kz = row["KZ"]
        edges = format_edges(row["wybrane odcinki"])
        decision = row["decyzja"]

        print(f"{pp:>3} | {lb:>6} | {kz:>4} | {edges:<35} |  
{decision}")

inf = float("inf")

# Macierz kosztów dla 6 miast.
matrix_6x6 = np.array([
    [inf, 25, 40, 31, 27, 15],
    [ 5, inf, 17, 30, 12, 20],
    [19, 22, inf, 11, 14, 8],
    [42, 13, 24, inf, 19, 33],
    [18, 10, 6, 25, inf, 21],
    [31, 7, 14, 9, 28, inf],
])

print("URUCHOMIENIE ALGORYTMU LITTLE'A")
print(f"Rozmiar macierzy: {matrix_6x6.shape[0]} x  
{matrix_6x6.shape[1]}")
print()

v_star, best_path, trace = little_algorithm(matrix_6x6)

print("KOLEJNE PODPROBLEMY:")
print_trace_table(trace)

print()
print("WYNIK KOŃCOWY:")
print(f"v* = {v_star:.1f}")
print(f"Wybrane odcinki: {best_path}")

```

```

path_dict = dict(best_path)
start_node = best_path[0][0]
ordered_nodes = [start_node]
current = start_node

for _ in range(len(best_path)):
    current = path_dict[current]
    ordered_nodes.append(current)

print("Pełna trasa:", " -> ".join(map(str, ordered_nodes)))

```

URUCHOMIENIE ALGORYTMU LITTLE'A

Rozmiar macierzy: 6 x 6

KOLEJNE PODPROBLEMY:

PP	LB	KZ	odcinki	decyzja
1	62.0	KZ0	-	podział
względem odcinka 2->1				
2	62.0	KZ0	2->1	podział
względem odcinka 5->3				
4	62.0	KZ0	2->1, 5->3	podział
względem odcinka 1->6				
6	63.0	KZ0	2->1, 5->3, 1->6	podział
względem odcinka 4->5				
8	63.0	KZ0	2->1, 5->3, 1->6, 4->5	podział
względem odcinka 3->4				
7	68.0	KZ0	2->1, 5->3	podział
względem odcinka 3->6				
11	68.0	KZ0	2->1, 5->3, 3->6	podział
względem odcinka 1->5				
13	68.0	KZ0	2->1, 5->3, 3->6, 1->5	podział
względem odcinka 4->2				
15	68.0	KZ0	2->1, 5->3, 3->6, 1->5, 4->2	podział
względem odcinka 6->4				
16	68.0	KZ0	2->1, 5->3, 3->6, 1->5	podział
względem odcinka 6->2				
17	68.0	KZ3	2->1, 5->3, 3->6, 1->5, 4->2, 6->4	zamknięto, bo otrzymano pełny cykl
18	68.0	KZ2	2->1, 5->3, 3->6, 1->5, 4->2	zamknięto, bo LB >= v*
19	68.0	KZ2	2->1, 5->3, 3->6, 1->5	zamknięto, bo LB >= v*
5	73.0	KZ2	2->1	zamknięto, bo LB >= v*
3	74.0	KZ2	-	zamknięto, bo LB >= v*
10	76.0	KZ2	2->1, 5->3, 1->6, 4->5	zamknięto, bo LB >= v*
14	76.0	KZ2	2->1, 5->3, 3->6	zamknięto,

```
bo LB >= v*
  9 |   78.0 | KZ2 | 2->1, 5->3, 1->6 | zamknięto,
bo LB >= v*
 12 |   89.0 | KZ2 | 2->1, 5->3 | zamknięto,
bo LB >= v*
```

WYNIK KOŃCOWY:

$v^* = 68.0$

Wybrane odcinki: [(1, 5), (5, 3), (3, 6), (6, 4), (4, 2), (2, 1)]

Pełna trasa: 1 -> 5 -> 3 -> 6 -> 4 -> 2 -> 1

Krótkie podsumowanie wyniku

Dla podanej macierzy kosztów otrzymano rozwiązanie optymalne o wartości $v^* = 68$.

Wyznaczona trasa ma postać:

1 -> 5 -> 3 -> 6 -> 4 -> 2 -> 1